

O'REILLY®

Fundamentals of DevOps and Software Delivery

A Hands-On Guide to Deploying and Managing Software in Production



Yevgeniy Brikman

Praise for *Fundamentals of DevOps and Software Delivery*

I'm not aware of any other book that provides such practical advice on operations and systems administration in the context of delivering and running software. Jim's book stays refreshingly focused on the real-world aspects of DevOps—what most people actually mean when they use the term—rather than the theory.

—Kief Morris, distinguished infrastructure engineer at Thoughtworks and author of *Infrastructure as Code*

An excellent step-by-step guide to getting practical things done—the kind of thing I wish I'd had when I was starting off.

—Niall Murphy, Co-founder & CEO of Stanza and co-author of *Site Reliability Engineering: How Google Runs Production Systems*

*Meticulously organized, chock full of examples, buoyed by research and practical experience, *Fundamentals of DevOps and Software Delivery* is another outstanding project from the prolific Yevgeniy “Jim” Brikman. The breadth of technology in this space can feel overwhelming, but Jim manages to isolate the signal from the noise, emphasizing the prevailing industry best practices while smartly identifying pitfalls along the way.*

—Ben Whaley, staff software engineer at Chime and co-author of *UNIX and Linux System Administration Handbook*

If you want to learn DevOps principles quickly, and learn them in a way that's directly applicable to your problems, buy this book.

—Yousif Akbar, principal software engineer, Gruntwork

Fundamentals of DevOps and Software Delivery

A Hands-On Guide to Deploying and Managing Software in Production

Yevgeniy Brikman

Fundamentals of DevOps and Software Delivery

by Yevgeniy Brikman

Copyright © 2025 Yevgeniy “Jim” Brikman. All rights reserved.

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Acquisitions Editor: John Devins

Development Editor: Melissa Potter

Production Editor: Christopher Faucher

Copyeditor: Sharon Wilkey

Proofreader: Emily Wydeven

Indexer: BIM Creatives, LLC

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Kate Dullea

May 2025: First Edition

Revision History for the First Edition

- 2025-05-20: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781098174590> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Fundamentals of DevOps and Software Delivery*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the author and do not represent the publisher's views. While the publisher and the author have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the author disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-098-17459-0

[LSI]

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

Dedication

To Mom, Dad, Lyalya, Molly, Flumpus, and Belka

Preface

There are many guides out there on how to write software. This book is a guide to *software delivery*—that is, all the processes, tools, and techniques that are required to run and maintain software in production on an ongoing basis. In particular, this book is a guide to *DevOps*, a methodology for making software delivery more efficient.

Whereas most books and talks on DevOps focus on culture, values, and organizational structure, I've found that when most people talk about DevOps, what they are really interested in is the mechanics of effective software delivery. That's what this book is all about. It's a *hands-on guide* that includes dozens of step-by-step examples of how to run production systems. You'll start with the basics—an app running on a single server—and work your way up to microservices in a Kubernetes cluster with a service mesh, automated deployment pipeline, end-to-end encryption, and more.

By the time you're done with the book, you will have had hands-on practice with all the core concepts and practices of modern DevOps and software delivery, including:

- Deploying VMs (EC2), containers (Kubernetes), and serverless apps (Lambda)
- Managing your infrastructure as code via OpenTofu, Packer, and Ansible
- Automating your builds, tests, and deployments in a CI/CD pipeline
- Configuring networking, including VPCs, VPNs, DNS, and service meshes
- Splitting your codebase into multiple environments, libraries, and microservices
- Managing secrets and encrypting data in transit (TLS) and at rest (AES)
- Storing data in relational databases, NoSQL databases, and file stores
- Setting up monitoring, including metrics, logs, events, and alerts
- And much more!

Before jumping into all this content, I want to take a moment to talk about why I felt the need to write this book—and why you may want to read it.

Why I Wrote This Book

Almost every piece of software depends on software delivery practices for deployment, maintenance, and security. And yet, I'm not aware of any hands-on guides that teach software delivery end to end. Just about everyone who learns software delivery today is learning it the hard way, through trial and error. Unfortunately, errors in software delivery can be costly: they result in outages, data loss, and security breaches. The lack of a good way to learn software delivery is making the entire software industry slower, less effective, and less secure.

I experienced this firsthand. In 2011, I was working at LinkedIn. From the outside, everything looked great: the company had just had its initial public offering (IPO), the share price was up by over 100%, revenue was growing by more than 100% year over year, and the website had over 100 million members, with 2 new members joining every single second. But from the inside, the company was in turmoil. Why? Because our software delivery practices had gotten so bad that we could no longer deploy.

Back then, we would do deployments once every two weeks, and it was always a painful, tedious, and error-prone affair. In 2011, we had a deployment that went so badly we couldn't complete it, no matter how hard we tried. We rolled out some new changes, which caused system instability; we pushed some fixes, but those caused new bugs; we pushed more fixes, but that only led to more issues. Teams worked through the night, into the next day, and we still couldn't get things stable. In the end, after a several-day deployment nightmare, we had to roll everything back.

Here was a company worth nearly \$10 billion, and we could not deploy code. To get out of this mess, we kicked off **Project Inversion**, which was a complete freeze on all new feature development for several months while the engineering, product, and design teams reworked all the underlying infrastructure, tooling, and practices. The result was a huge success: months later, we were able to deploy dozens of times per day, with far fewer issues and outages, and that allowed the whole company to move faster.

Today, we might call this a "DevOps transformation" (though back then, the term "DevOps" had just appeared on the scene, so we didn't call it that), and to get there, we had to go through a lot of pain and outages. The truth is, we didn't know what we didn't know. We had to go out and chat with companies across the industry, learning about trunk-based development from one company, canary deployments from another, feature toggles from another, and so on.

Sadly, even now, as I write this book nearly 15 years later, relatively few developers know about these DevOps and software delivery practices. After leaving LinkedIn, I cofounded **Gruntwork**, where I had the opportunity to work with hundreds of companies on their DevOps and software delivery practices. What I saw was LinkedIn's DevOps nightmare repeated over and over again at companies of all sizes. The techniques that a handful of the top tech companies had figured out were not filtering down to the rest of the industry. Most developers out there still don't know what they don't know.

So I decided to write a book. I hope that this book can be a small step in improving this situation. I hope that a comprehensive, hands-on overview of DevOps and software delivery will help the next generation of software companies get off on the right foot and avoid some of the DevOps nightmares I've seen. I hope that instead of just hacking things together and learning things the hard way, this book will allow you to learn from the experience of others.

Perhaps the result will be a software industry that can build software faster, more reliably, and more securely. If that happens, it could have a profound impact on the industry, as described next.

The Impact of World-Class Software Delivery

The vast majority of developers have never had the opportunity to see what world-class software delivery looks like firsthand. If you're one of them, you'll be astonished by the gap between companies with world-class software delivery processes and everyone else. It's not a 1.1 times or 1.5 times improvement: it's 10 times, 100 times, or more.

Table P-1 shows the difference between elite performers and low performers in the four key *DevOps Research and Assessment (DORA) metrics*, which are a quick way to assess the performance of a software development team.

Table P-1. DORA metrics performance from the 2024 State of DevOps Report

Metric	Description	Elite vs. low performers
Deployment frequency	How often you deploy to production	182× more often
Lead time	How long it takes a change to go from committed to deployed	127× faster
Change failure rate	How often deployments cause failures that need immediate remediation	8× lower
Recovery time	How long it takes to recover from a failed deployment	2,293× faster

To put these *staggering* differences into perspective, consider these examples:

- Deploying once per month versus many times per day
- Deployment processes that take 36 hours versus 5 minutes
- Two out of three deployments causing problems versus one out of twenty

- Outages that last 24 hours versus 2 minutes

It's almost a meme that developers who leave companies with world-class software delivery processes, such as Google, Meta, Amazon, or LinkedIn (after Project Inversion), complain bitterly about how much they miss the infrastructure and tooling. That's because they are used to a world that looks like this:

- They can deploy anytime they want, even *thousands of times per day*.
- Deployments can happen in *minutes*, and they are 100% automated.
- Problems can be detected in *seconds*, often before any user-visible impact.
- Outages can be resolved in *minutes*, often automatically.

What do the equivalent numbers look like at your organization? If you're not even in the ballpark, don't fret. The first thing you need to know is that it's possible to achieve these results, even if you're not a multibillion-dollar company. In fact, there are many ways to achieve these sorts of results, and each of those world-class companies does it a bit differently. That said, they also have a lot in common, and DevOps is an attempt to capture some of the common patterns.

Where DevOps Came From

DevOps emerged in the late 2000s as a response to inefficiencies in the way companies delivered software. Historically, most companies had a Developer (*Dev*) team responsible for writing the software, and an Operations (*Ops*) team responsible for managing the hardware. In many companies, these teams worked in silos with conflicting objectives. The Ops team's core objectives were usually security and reliability, so they were the only ones with access to production systems, and were on the hook for doing deployments and dealing with outages. The Dev team's core objectives were usually to ship features and to do so as quickly as possible. This arrangement often led to problems.

The Dev team, which was typically working under time pressure, with no access to production systems or on-call duties, would write the code, and with little testing, "toss it over the wall" to the Ops team. The Ops team would then manually deploy that code, which frequently led to failures or outages, either from an error in the deployment process or a bug in the code. The Ops team, tired of waking up at 3 a.m. to deal with outages, would end up reducing the release cadence. Unfortunately, this led to a vicious cycle: fewer releases meant each release was bigger and more complicated, which led to more outages, which led to an even slower release cadence, and so on, until the company would grind to a halt.

Although the problems with manual processes and siloing were known for many years, it wasn't until around 2007–2008 that the software industry started to discuss these issues more openly in various

conferences, meetups, and articles. These discussions coincided with, and were big drivers of, several major shifts that happened over the next decade:

The shift to the cloud

Instead of managing their own data centers, many companies started moving to the cloud, renting servers and other infrastructure from third-party providers (you'll learn all about the cloud in **Chapter 1**). This represented a big shift for both the Ops team, which was suddenly spending most of its time dealing with software (from the cloud provider) instead of hardware (e.g., racking servers and plugging in network cables), and the Dev team, which could now access production systems through the same software tools that Ops was using.

The shift to automation

Instead of doing everything manually, many companies started to automate their processes via configuration management tools (**Chapter 2**), orchestration tools (**Chapter 3**), automated testing (**Chapter 4**), and continuous integration/continuous delivery (**Chapter 5**). These automations were another big shift for both Dev and Ops, as they defined a new set of interaction points between the two teams (a bit like an API).

The shift to cross-functional teams

Instead of working in silos, many companies started to have Dev and Ops work more closely together on *cross-functional teams* with shared objectives and shared tooling (such as the cloud and automation tooling).

All these trends together became known as the *DevOps movement*. DevOps isn't the name of a team or a job title or a particular technology (though you'll see it used for all three). Instead, it's a set of processes, ideas, and techniques. Everyone has a slightly different definition of DevOps, but for this book, I'm going to define DevOps as a methodology with the following goal:

The goal of DevOps is to make software delivery vastly more efficient.

You read how LinkedIn's DevOps transformation saved the company, but there are many other examples. Nordstrom found that applying DevOps practices increased the number of features it delivered per month by 100%, reduced defects by 50%, reduced lead times by 60%, and reduced the number of production incidents by 60% to 90%. After HP's LaserJet Firmware division began using DevOps practices, the amount of time its developers spent on developing new features increased from 5% to 40%, and overall development costs were reduced by 40%. Etsy used DevOps practices to go from stressful, infrequent deployments that caused numerous outages to deploying 25 to 50 times per day, with fewer outages.¹

All that said, a fair warning: while the results from adopting DevOps can be wonderful, the experience along the way can be anything but wonderful, as described next.

Watch Out for Snakes

I'm going to let you in on a little secret: we use a single word, "DevOps," to describe what's actually dozens and dozens of largely unrelated concepts. What does the cryptography behind a TLS certificate have to do with defining a deployment pipeline in GitHub Actions YAML or backing up data from a PostgreSQL database? Not much. And yet, your typical site reliability engineer (SRE) or DevOps engineer has to deal with all of these, and countless other concepts too.

What makes DevOps hard is not that any one of these concepts is incredibly complicated by itself, but that there are so many concepts to master—and you have to connect them all together *just right*. The TLS certificate must be configured just right, or your users will get scary errors that prevent them from accessing your website. Your deployment pipeline must be configured just right, or your team won't be able to deploy. Your database backup must be set up just right, or you are at risk of data loss, and if you lose all your data, you may go out of business entirely. DevOps is a remarkable combination of an incredibly broad surface area, but also one requiring you to sweat every single detail—for either you get everything connected together correctly, or nothing works at all.

I often use the analogy of a box of cables: you reach into the box, hoping to pull out just one cable, but you inevitably end up pulling out a giant, tangled mess. Unfortunately, that's the state of DevOps today. It's a relatively new industry, the tools and techniques we have aren't that mature, and it often feels like everything is broken and frustrating and hopelessly tangled.

My hope in this book is, as much as I can, to untangle this mess of cables for you. To show you that these are, in fact, separate cables—separate concepts—that, in isolation, are something you can readily understand, begin to work with, and ultimately become proficient in.

But sometimes this is hard to do. Sometimes, reaching into this box of cables feels more like reaching into a box of snakes. You end up getting bitten. If you find yourself sitting there, staring at a nonsense error message, tearing your hair out, stressed, angry, and afraid, know this: you are not alone.

Thousands of other developers are reaching into that box of snakes and getting bitten, every day. I'm one of them. I've lost more hair to DevOps than I care to admit. Even while writing this book, I frequently found myself frustrated or confused or yelling at my screen, even though I've done most of these things a thousand times. That's just how it is today.

In fact, in a few places in this book, I haven't been able to untangle the wires as much as I'd like. For instance, some example code was just too complicated and long to include in the book, so I have to settle for a simpler and less realistic version. Or I can't explain a certain concept without introducing 10 other concepts that come later, so I can give you only a partial explanation for now. In cases like these, I've added a "box of snakes" warning that looks like this:

WATCH OUT FOR SNAKES

When you see one of these warnings, be prepared to enter a particularly hairy and tangled corner of DevOps.

Whenever you see such a warning, understand that you're going to see part of the picture now but perhaps won't be able to get the full picture until later. This is true of DevOps in general. If you're new to DevOps, the content will initially seem strange, confusing, and full of incomprehensible buzzwords. And each time you go to learn a new buzzword, you're hit with 10 more unfamiliar buzzwords, so you never feel like you're getting the whole picture. But I promise that if you give it enough time, you'll eventually get over a hump, and suddenly, the pieces will start to make sense and really come together. You need to build up a big enough base of knowledge and experience, and it's hard going at first, but at some point, it starts to get easier. DevOps never becomes completely easy, but you get to a point where you always feel confident that you can figure it out.

So stick with it. And watch out for those snakes.

Who Should Read This Book

This book is for anyone responsible for deploying and managing apps in production—that is, anyone responsible for software delivery. This includes the following:

Individual contributors in operations roles

Current and aspiring SREs, DevOps engineers, sysadmins, operations engineers, and release engineers who want to level up their knowledge of software delivery.

Individual contributors in dev roles

Software engineers, software developers, web developers, and full stack engineers who want to learn more about the operations side of the house.

Managers

Engineering managers, engineering directors, chief technology officers (CTOs), vice presidents of engineering (VPEs), and chief information officers (CIOs) who want to learn how to adopt DevOps practices in their organizations.

This book does not assume that you're already an expert coder or expert sysadmin. A basic familiarity with programming, the command line, and server-based software (e.g., websites) should suffice. Everything else you need, you'll be able to pick up as you go. The only tools you need are a computer, an internet connection, and the desire to learn.

What You'll Find in This Book

Table P-2 shows a chapter-by-chapter outline of the book's content, including the key ideas you'll explore and the hands-on examples you'll try in each chapter.

Table P-2. An outline of the book

Chapter	Key ideas you'll explore	Examples you'll try out
Chapter 1, "How to Deploy Your App"	• Personal computers versus servers • On-prem versus cloud • PaaS versus IaaS	• Run an app locally • Run an app on Render • Run an app on an EC2 instance in AWS
Chapter 2, "How to Manage Your Infrastructure as Code"	• Ad hoc scripts • Configuration management tools • Server templating tools • Provisioning tools	• Use Bash to deploy an EC2 instance • Use Ansible to deploy an EC2 instance • Use Packer to build an AMI • Use OpenTofu to deploy an EC2 instance
Chapter 3, "How to Manage Your Apps by Using Orchestration Tools"	• Server orchestration • VM orchestration • Container orchestration • Serverless orchestration	• Use Ansible to deploy app servers and nginx • Use OpenTofu to deploy an ASG and ALB • Deploy a Dockerized app in Kubernetes • Deploy a serverless app with AWS Lambda

Chapter 4, “How to Version, Build, and Test Your Code”

- Version control
- Build systems
- Dependency management
- Automated testing
- Store your code in GitHub
- Configure your build in npm
- Set up automated tests for a Node.js app
- Set up automated tests for OpenTofu code

Chapter 5, “How to Set Up Continuous Integration and Continuous Delivery”

- Trunk-based development
- Feature toggles
- Deployment strategies, pipelines
- Use OIDC with GitHub Actions and AWS
- Run tests in GitHub Actions
- Run deployments in GitHub Actions

Chapter 6, “How to Work with Multiple Teams and Environments”

- Multiple environments
- Multiple libraries
- Multiple services
- Create multiple AWS accounts
- Configure apps for multiple environments
- Deploy microservices in Kubernetes

Chapter 7, “How to Set Up Networking”

- Domain Name System (DNS)
- Virtual private clouds (VPCs)
- Network access and hardening
- Service discovery, service meshes
- Set up a custom domain name in Route 53
- Deploy a custom VPC in AWS
- Use SSH to connect to a server
- Use Istio as a service mesh with Kubernetes

Chapter 8, “How to Secure
Communication and Storage”

- Cryptography
- Encryption at rest
- Encryption in transit
- Encrypt data with AES and RSA

This document was truncated here because it was created in the Evaluation Mode.

